CHAPTER 02

IAM & Security Foundations

Users · Roles · KMS · Secrets Manager · Identity Federation

02-00 Chapter Overview

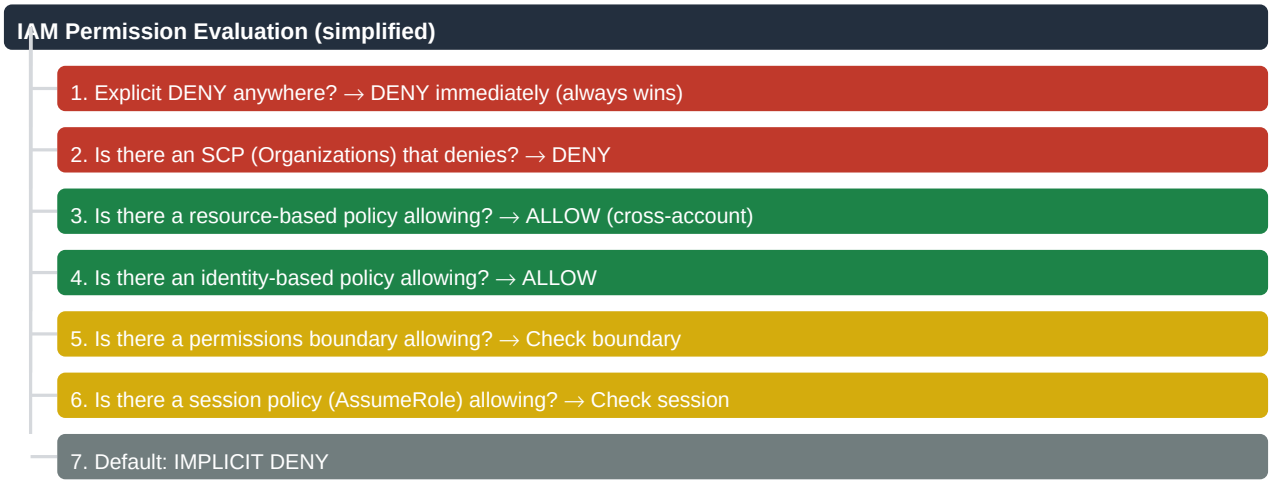
Topic	AWS Service	ShopFlow Use-Case
IAM Users, Groups & Roles	IAM	Developer access, service accounts
IAM Policies — inline vs managed	IAM Policies	Least privilege for ECS tasks
STS & Assume Role	STS, AssumeRole, OIDC	Cross-account CI/CD pipeline
Secrets Management	Secrets Manager, Parameter Store	DB passwords, Stripe API keys
Encryption — KMS	KMS, CMK, Data Keys	Encrypt RDS, S3, SQS at rest
MFA & Password Policy	IAM, Virtual MFA, Hardware MFA	Admin account protection
Identity Federation	IAM Identity Center, SAML, OIDC	Google Workspace SSO for devs
Audit & Compliance	CloudTrail, IAM Access Analyzer	PCI-DSS audit trail

02-01 IAM Fundamentals — Users, Groups & Roles

AWS Identity and Access Management (IAM) controls who can do what with AWS resources. It is global (not region-specific), free, and the single most important service to master. ShopFlow uses IAM roles exclusively — no long-lived user access keys in production.

IAM Principal Types

Principal Type	Description	ShopFlow Usage	Credential Type
IAM User	Long-lived identity for a person or system	Engineers via SSO only (no direct user)	Password + MFA or access key (avoided)
IAM Group	Collection of IAM users sharing policies	Developers, Ops, ReadOnly groups	N/A — groups have no credentials
IAM Role	Temporary identity assumed by a service/user	ECS tasks, Lambda, CI/CD, cross-account	Temporary STS credentials (15min-36hr)
AWS Service Principal	AWS service acting on your behalf	RDS enhanced monitoring, ECS agent	Managed by AWS automatically
Federated Identity	External IdP users mapped to IAM roles	Google Workspace SSO via IAM Identity Ctr	SAML assertion → STS temp creds



■ Explicit DENY always overrides any ALLOW — even if 10 policies allow an action, one explicit DENY anywhere in the chain blocks it.

02-02 IAM Policies — Writing Least-Privilege Permissions

IAM policies are JSON documents that define permissions. Policy evaluation follows a default-deny model: everything is denied unless explicitly allowed. Writing least-privilege policies is the most impactful security practice.

Policy Structure

```
{
"Version": "2012-10-17",
"Statement": [
{
"Sid": "AllowProductServiceS3Read",
"Effect": "Allow",
"Action": [
"s3:GetObject",
"s3:ListBucket"
],
"Resource": [
"arn:aws:s3:::shopflow-images",
"arn:aws:s3:::shopflow-images/*"
],
"Condition": {
"StringEquals": {"aws:RequestedRegion": "us-east-1"}
}
}
]
}
```

Policy Element	Description	Best Practice
Effect	Allow or Deny the actions listed	Use Allow; add explicit Deny for critical restrictions
Action	Which AWS API calls are permitted	Be specific: s3:GetObject not s3:* not *
Resource	Which ARNs the action applies to	Be specific: arn:aws:s3:::bucket/prefix/* not *
Condition	Additional constraints (IP, MFA, tags, region)	Add aws:RequestedRegion + aws:PrincipalTag conditions
Principal	Who this policy applies to (resource-based)	In trust policies: specify exact ARN, not *, add conditions

Policy Type	Attached To	Scope	ShopFlow Use
AWS Managed	IAM User/Group/Role	Predefined by AWS, broad	ReadOnlyAccess for audit users

Policy Type	Attached To	Scope	ShopFlow Use
Customer Managed	IAM User/Group/Role	Custom, reusable across roles	ProductServicePolicy, OrderServicePolicy
Inline	Single IAM entity	One-to-one, deleted with entity	Emergency break-glass permissions
Resource-Based	AWS Resource (S3, SQS, KMS)	Cross-account grants	S3 bucket policy for CloudFront OAC
Permissions Boundary	IAM User/Role	Max permissions ceiling	Limit developer-created roles
SCP	AWS Organization OU/Account	Guardrails across all principals	Deny outside approved regions

■ Use IAM Access Analyzer to generate least-privilege policies from CloudTrail activity. It shows exactly which permissions a role has actually used in the last 90 days.

02-03 IAM Roles for AWS Services — ECS & Lambda

In ShopFlow, every ECS task and Lambda function runs with a dedicated IAM role that grants only the permissions it needs. This is the single most effective way to limit blast radius if a service is compromised.

ECS Task Role — Product Service

```
# CDK: Create product-svc task role
const productTaskRole = new iam.Role(this, "ProductTaskRole", {
assumedBy: new iam.ServicePrincipal("ecs-tasks.amazonaws.com"),
roleName: "shopflow-product-svc-task-role",
description: "Minimum permissions for product microservice",
});
// Grant read-only access to product images S3 bucket
productImagesBucket.grantRead(productTaskRole);
// Grant read/write to product cache (ElastiCache via VPC, no IAM)
// Grant send to product events SQS queue
productEventQueue.grantSendMessages(productTaskRole);
// Grant read-only to product secrets in Secrets Manager
productDbSecret.grantRead(productTaskRole);
```

Lambda Execution Role — Thumbnail Generator

```
const thumbnailRole = new iam.Role(this, "ThumbnailLambdaRole", {
assumedBy: new iam.ServicePrincipal("lambda.amazonaws.com"),
managedPolicies: [
iam.ManagedPolicy.fromAwsManagedPolicyName(
"service-role/AWSLambdaVPCAccessExecutionRole")
],
});
// Only read from raw-images, write to processed-images
rawImagesBucket.grantRead(thumbnailRole);
processedImagesBucket.grantWrite(thumbnailRole);
```

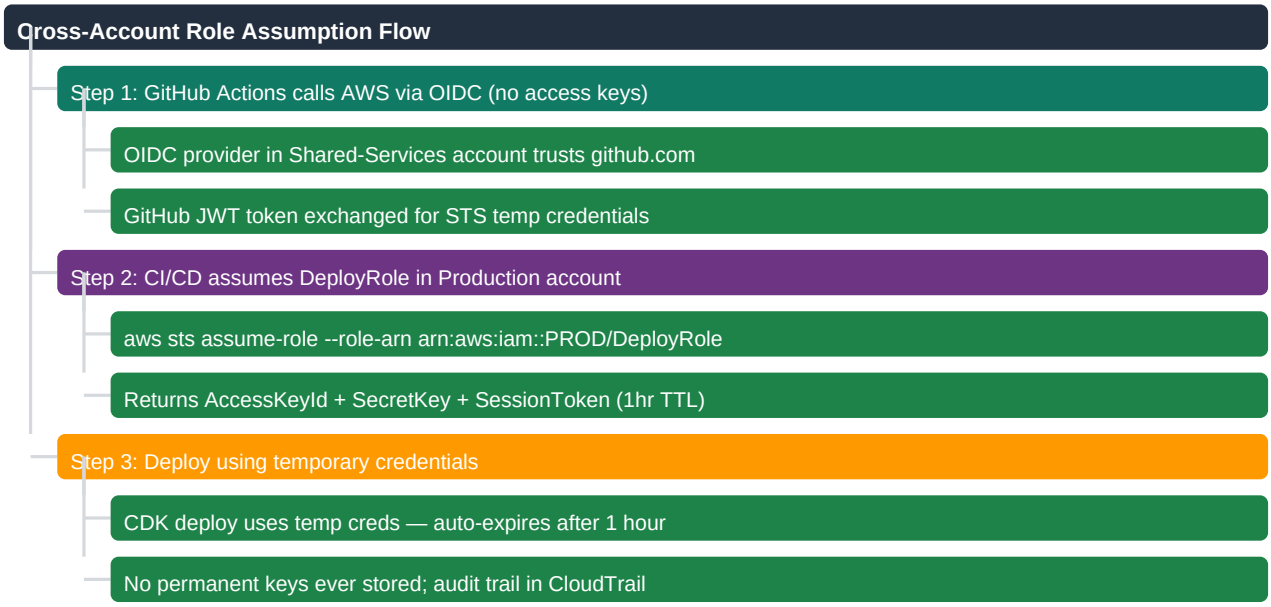
Service	Role Principal	Common Permissions	What to NEVER grant
ECS Fargate task	ecs-tasks.amazonaws.com	Specific S3 bucket, SQS queue, Secrets Manager	s3:* on any resource
Lambda function	lambda.amazonaws.com	Specific trigger source, output destination	iam:* (privilege escalation risk)
CodeBuild	codebuild.amazonaws.com	ECR push/pull, CloudFormation deploy	AdministratorAccess
RDS monitoring	monitoring.rds.amazonaws.com	CloudWatch metrics only (managed policy)	Any application-level permissions

■ Use the principle of least privilege. If your product-svc only reads S3, grant s3:GetObject on the specific bucket prefix. If it is compromised, the attacker can only read product images — not orders, payments, or secrets.

02-04 AWS STS & Cross-Account Access

AWS Security Token Service (STS) issues short-lived credentials when a principal assumes a role. This is the mechanism behind all cross-account access in ShopFlow (e.g., CI/CD pipeline in Shared Services account deploying to Production account).

AssumeRole Flow — CI/CD Cross-Account Deploy



Production Account Trust Policy for CI/CD Role

```
// Production account DeployRole trust policy
{
"Effect": "Allow",
"Principal": {
"AWS": "arn:aws:iam::SHARED_SERVICES_ACCT:role/CodeBuildRole"
},
"Action": "sts:AssumeRole",
"Condition": {
"StringEquals": {
"sts:ExternalId": "shopflow-cicd-deploy-2024"
}
}
}
```

STS Feature	Use Case	TTL	ShopFlow Usage
AssumeRole	Cross-account, cross-service access	15min - 12hr	CI/CD deploying to prod
AssumeRoleWithWebIdentity	OIDC (GitHub Actions, EKS pods)	1hr default	GitHub Actions CI pipeline
AssumeRoleWithSAML	Enterprise SSO (Okta, Google)	1hr max	Developer console access
GetSessionToken	MFA-protected API calls	15min - 36hr	Break-glass admin operations
GetFederationToken	Legacy federated applications	15min - 36hr	Avoided in ShopFlow

■ Always use OIDC for GitHub Actions. It eliminates the need for stored AWS access keys entirely. ShopFlow CI/CD has zero long-lived secrets in GitHub repository settings.

02-05 AWS Secrets Manager & Parameter Store

ShopFlow has 23 secrets: database passwords, Stripe API keys, OAuth client secrets, JWT signing keys, and third-party API tokens. These must never appear in code, Dockerfiles, or environment variables in plain text.

Secrets Manager vs Systems Manager Parameter Store

Feature	Secrets Manager	SSM Parameter Store	ShopFlow Choice
Pricing	$0.40/secret/month + API calls	Free (Standard), $0.05/param/month (Advanced)	Secrets Manager for secrets, SSM for config
Automatic rotation	Built-in Lambda rotation for RDS, Redshift, DocumentDB	Custom Lambda required	Secrets Manager for DB passwords
Cross-account access	Via resource-based policy	Via assume role	Secrets Manager
Secret size	Up to 65KB (JSON blob)	4KB Standard, 8KB Advanced	Both adequate
Versioning	Full version history with AWSCURRENT, AWSPREVIOUS labels	Yes, unlimited versions	Both support
AWS integrations	RDS, Redshift, DocumentDB native rotation	ECS, Lambda, CodeBuild env injection	Both integrated

ShopFlow Secrets Architecture

```
# Secret naming convention: shopflow/{env}/{service}/{name}
shopflow/prod/order-svc/aurora-password
shopflow/prod/order-svc/stripe-secret-key
shopflow/prod/user-svc/google-oauth-client-secret
shopflow/prod/notify-svc/sendgrid-api-key
# Retrieve secret in Java (ECS task role grants secretsmanager:GetSecretValue)
SecretsManagerClient client = SecretsManagerClient.builder()
.region(Region.US_EAST_1).build();
GetSecretValueResponse resp = client.getSecretValue(
GetSecretValueRequest.builder()
.secretId("shopflow/prod/order-svc/aurora-password")
.build());
String dbPassword = resp.secretString();
```

Automatic Secret Rotation — Aurora PostgreSQL

```
# CDK: Enable 30-day rotation for Aurora password
const dbSecret = new secretsmanager.Secret(this, "AuroraSecret", {
secretName: "shopflow/prod/order-svc/aurora-password",
generateSecretString: {
secretStringTemplate: JSON.stringify({username:"shopflow_app"}),
generateStringKey: "password",
excludePunctuation: true,
passwordLength: 32,
},
});
dbSecret.addRotationSchedule("Rotation", {
automaticallyAfter: Duration.days(30),
hostedRotation: secretsmanager.HostedRotation.postgreSqlSingleUser({
vpc, vpcSubnets: {subnetType: ec2.SubnetType.PRIVATE_WITH_EGRESS}
}),
});
```

■ Never pass secrets as ECS environment variables in plain text — they appear in CloudTrail and ECS task metadata API. Use Secrets Manager injection via task definition secrets field instead.

02-06 AWS KMS — Encryption Key Management

AWS Key Management Service (KMS) manages encryption keys for all ShopFlow data at rest. KMS integrates natively with S3, RDS, DynamoDB, EBS, SQS, SNS, CloudWatch Logs, and Secrets Manager.

KMS Key Types

Key Type	Description	Use Case	Cost
AWS Managed Key (aws/s3)	Created & managed by AWS per service	Default encryption, no extra config	Free (AWS manages)
Customer Managed Key (CMK)	You create, control rotation, set policy	Compliance, cross-account, custom key policies	$1/key/month + $0.03/10K API calls
External Key (BYOK)	Your key material imported into KMS	Specific compliance requirement	$1/key/month
Custom Key Store (CloudHSM)	Keys in dedicated HSM hardware	Highest compliance (FIPS 140-2 Level 3)	$1.45/hr/HSM

ShopFlow KMS Key Setup

```
# CDK: Create CMK for ShopFlow production data
const shopflowKey = new kms.Key(this, "ShopFlowDataKey", {
alias: "alias/shopflow/prod/data",
description: "ShopFlow production data encryption key",
enableKeyRotation: true, // annual auto-rotation
removalPolicy: RemovalPolicy.RETAIN,
});
// Grant Aurora to use key for encryption
shopflowKey.grantEncryptDecrypt(auroraCluster.clusterEndpoint);
// S3 bucket encrypted with same CMK
new s3.Bucket(this, "ShopFlowImages", {
encryption: s3.BucketEncryption.KMS,
encryptionKey: shopflowKey,
blockPublicAccess: s3.BlockPublicAccess.BLOCK_ALL,
});
```

Envelope Encryption — How KMS Protects ShopFlow Data

KMS never stores or sends the data key in plaintext outside of AWS. Instead it uses envelope encryption:

- 1. ShopFlow requests a data key from KMS (GenerateDataKey API call)
- 2. KMS returns: plaintext data key (use to encrypt your data) + encrypted data key
- 3. ShopFlow encrypts the data with the plaintext key, then discards the plaintext key
- 4. Stores the encrypted data + encrypted data key together (KMS never sees your data)
- 5. To decrypt: call KMS Decrypt with the encrypted data key → get plaintext key → decrypt data

■ Enable key rotation annually on all CMKs. KMS keeps old key material for decryption of existing data but uses the new key for all new encryptions. Zero downtime during rotation.

02-07 MFA, Password Policy & Account Hardening

Account hardening reduces the attack surface of ShopFlow's AWS accounts. These settings should be applied within the first 30 minutes of account creation.

Root Account Protection — Critical

- Enable MFA on root account immediately (virtual MFA or hardware MFA device)
- Delete all root account access keys (IAM console → Security credentials)
- Set a complex root password and store in 1Password/Vault (never use it again)
- Enable CloudTrail alert for root login: SNS notification to security team Slack

IAM Password Policy for ShopFlow

```
aws iam update-account-password-policy \
--minimum-password-length 16 \
--require-uppercase-characters \
--require-lowercase-characters \
--require-numbers \
--require-symbols \
--allow-users-to-change-password \
--max-password-age 90 \
--password-reuse-prevention 12 \
--hard-expiry
```

MFA Enforcement via IAM Policy

```
// Force MFA for all actions except MFA setup
{"Sid":"DenyWithoutMFA","Effect":"Deny",
"NotAction":["iam:CreateVirtualMFADevice",
"iam:EnableMFADevice",
"sts:GetSessionToken"],
"Resource":"*",
"Condition":{"BoolIfExists":
{"aws:MultiFactorAuthPresent":"false"}}}
```

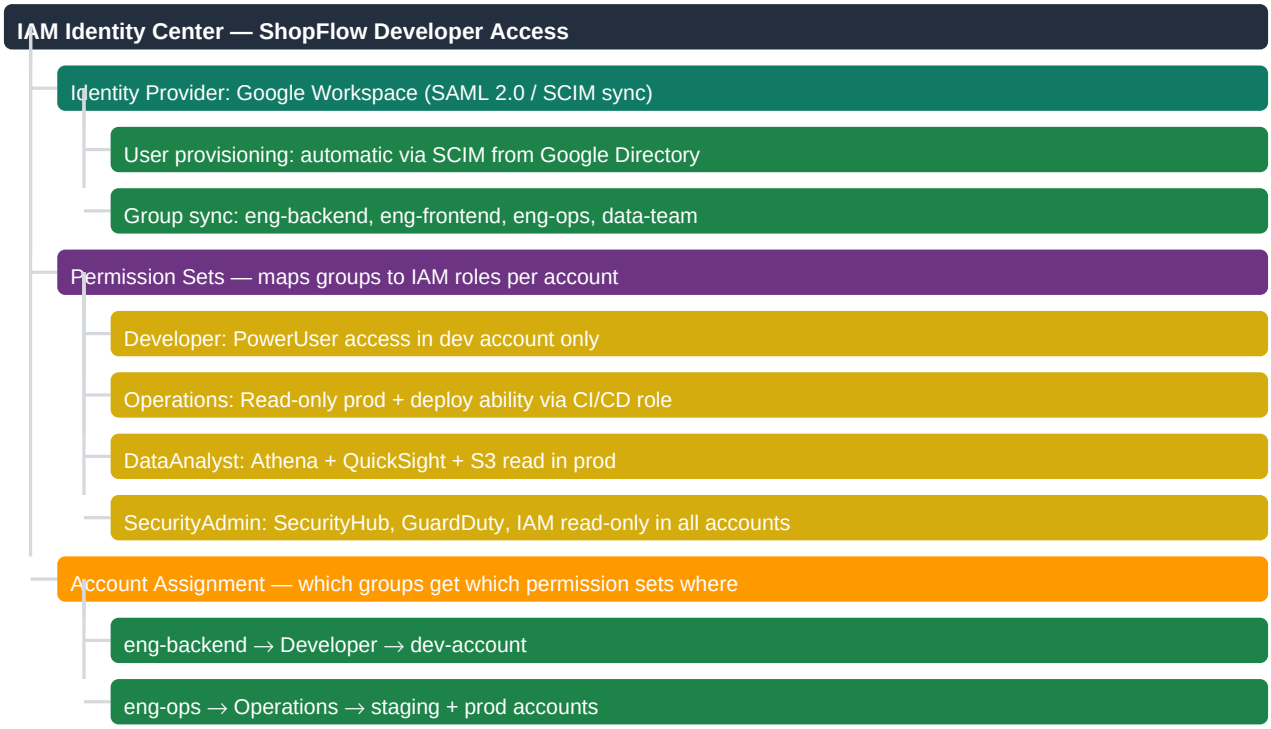
Hardening Control	Priority	Implementation	ShopFlow Status
Root account MFA	Critical	Hardware MFA device in safe	Done — Day 1
Delete root access keys	Critical	IAM Security credentials tab	Done — Day 1
CloudTrail all regions	Critical	CloudFormation StackSet from security account	Done
GuardDuty enabled	High	Enable in Security account, delegate to org	Done
Config enabled	High	All regions, all resource types	Done
Security Hub enabled	High	Aggregate findings from all accounts	Done
IAM Access Analyzer	High	One analyzer per account, one per org	Done
AWS Config required-tags rule	Medium	Alert on untagged resources after 24hr	In Progress

■ Enable AWS Security Hub to get a consolidated security score. ShopFlow targets 90+ score in Security Hub. Common findings: public S3 buckets (critical), CloudTrail disabled (critical), unrestricted SG (high).

02-08 IAM Identity Center — Developer SSO

AWS IAM Identity Center (formerly SSO) is the recommended way for ShopFlow engineers to access the AWS console and CLI. It connects to Google Workspace (IdP), provisions IAM roles automatically, and issues short-lived credentials.

SSO Setup Architecture



CLI Access with SSO

```
# Login (opens browser)
aws sso login --profile shopflow-dev
# Switch to prod account (read-only)
aws sso login --profile shopflow-prod-readonly
# Check current identity
aws sts get-caller-identity --profile shopflow-dev
# Output:
# { "UserId": "AROAXXX:manish@shopflow.com",
# "Account": "123456789012",
# "Arn": "arn:aws:sts::...:assumed-role/Developer/manish@shopflow.com" }
```

■ Tokens from aws sso login expire in 8 hours by default. Set sso_session_duration = 12h in ~/.aws/config for longer sessions. Credentials are stored encrypted in OS keychain.

02-09 IAM Access Analyzer & Permission Auditing

IAM Access Analyzer identifies resources shared with external principals, generates least-privilege policies from CloudTrail logs, and validates policies before deployment.

Access Analyzer Features for ShopFlow

Feature	What It Does	ShopFlow Alert Threshold
External access analysis	Finds resources accessible outside your account	Alert on any public S3 or cross-account SQS
Unused access findings	IAM users/roles with unused permissions (>90 days)	Remove permissions unused >90 days
Policy generation	Analyzes CloudTrail, generates minimum IAM policy	Generate policy for all new services
Policy validation	Checks policies for errors and security warnings	Block merge if validation fails in CI

Generate Least-Privilege Policy from CloudTrail

```
# Start policy generation (analyzes last 90 days of CloudTrail)
aws iam generate-service-last-accessed-details \
--arn arn:aws:iam::123456789012:role/ProductTaskRole
# Get generated details
aws iam get-service-last-accessed-details \
--job-id abc123
# Shows: which services were used, when, and from which IAM actions
# Or use Access Analyzer policy generator in the console:
# IAM → Access Analyzer → Policy generation → Select role → Generate
```

CloudTrail — IAM Event Monitoring

```
# Find all actions taken by root account (should be 0)
aws cloudtrail lookup-events \
--lookup-attributes AttributeKey=Username,AttributeValue=root \
--start-time 2024-01-01 --end-time 2024-12-31
# Find all IAM role creations in the last 7 days
aws cloudtrail lookup-events \
--lookup-attributes AttributeKey=EventName,AttributeValue=CreateRole \
--start-time $(date -d "7 days ago" --iso-8601)
```

■ Set up CloudWatch Metric Filter + Alarm for: root account login, IAM policy changes, security group changes, and CloudTrail disabled. These four alarms are PCI-DSS Requirement 10.6 and detect 80% of common attack patterns.

02-10 ShopFlow IAM Architecture — Full Design

02-11 Permission Boundaries — Delegating Safely

Permissions Boundaries prevent privilege escalation when granting developers the ability to create IAM roles. Without a boundary, a developer with iam:CreateRole could create a role with AdministratorAccess and assume it.

ShopFlow Developer Boundary Policy

```
// Permission boundary: developers can create roles, but only
// within these allowed services (no IAM, no Organizations)
{
"Version": "2012-10-17",
"Statement": [
{
"Effect": "Allow",
"Action": [
"s3:*", "sqs:*", "sns:*", "lambda:*",
"ec2:Describe*", "logs:*", "cloudwatch:*"
],
"Resource": "*"
},
{
"Effect": "Deny",
"Action": ["iam:*", "organizations:*", "account:*"],
```

```
"Resource": "*"
}
]
}
```

Attaching Boundary When Developers Create Roles

```
// Developer's CDK code — boundary enforced via SCP
const devRole = new iam.Role(stack, "FeatureRole", {
assumedBy: new iam.ServicePrincipal("lambda.amazonaws.com"),
permissionsBoundary: iam.ManagedPolicy.fromManagedPolicyArn(
stack,
"DeveloperBoundary",
`arn:aws:iam::${account}:policy/shopflow-developer-boundary`
),
});
// If developer forgets the boundary, SCP blocks role creation
```

■ Without permission boundaries, any developer with iam:CreateRole and iam:PassRole can escalate to administrator. Boundaries + SCPs together prevent this.

02-12 Security Incident Response — IAM Playbook

When an IAM credential is compromised, speed matters. ShopFlow has a documented runbook for credential incidents that can be executed in under 10 minutes.

Credential Compromise Response Runbook

Step	Action	AWS Command	Time Target
1. Detect	GuardDuty alert or suspicious CloudTrail event	GuardDuty console / Findings	Automated (0 min)
2. Contain	Attach DenyAll policy to compromised role	aws iam put-role-policy	<5 minutes
3. Revoke sessions	Revoke all active sessions for the role	aws iam delete-role-policy + update policy	<5 minutes
4. Rotate credentials	Delete and create access keys if user, delete access key	aws iam delete-access-key	<10 minutes
5. Investigate	Pull CloudTrail events for the last 24 hours for the principal	aws cloudtrail lookup-events	15-30 minutes
6. Remediate	Remove unauthorized resources created, audit blast radius	Manual review	30-60 minutes
7. Notify	Security team + management + Legal if PII exposed	Email / PagerDuty	<5 minutes

Emergency Credential Revocation Commands

```
# Step 1: Deny all actions immediately
aws iam put-role-policy \
--role-name CompromisedRole \
--policy-name EMERGENCY-DENY-ALL \
--policy-document '{"Version":"2012-10-17",
"Statement":[{"Effect":"Deny","Action":"*",
"Resource":"*"}]}'
# Step 2: Revoke active sessions
aws iam update-assume-role-policy \
--role-name CompromisedRole \
--policy-document file://deny-all-trust.json
# Step 3: Pull CloudTrail events for investigation
aws cloudtrail lookup-events \
--lookup-attributes AttributeKey=Username,
AttributeValue=CompromisedRole \
--start-time $(date -d "24 hours ago" --iso-8601)
```

■ GuardDuty automatically detects: leaked credentials used from unusual IP/geography, S3 bucket enumeration, EC2 crypto-mining. Enable it in every account — it costs ~$3/month for a small account.

02-BP Best Practices & Anti-Patterns

✓ Use IAM Identity Center (SSO) for all human access — no direct IAM users for developers

✓ Assign a dedicated IAM role to every ECS task and Lambda function

✓ Use Secrets Manager with automatic rotation for all database passwords

✓ Enable MFA for root account and all privileged IAM users immediately

✓ Use Permissions Boundaries when delegating role creation to developers

✓ Enforce least privilege: start with no permissions, add only what is needed

✓ Enable IAM Access Analyzer in every account — alerts on external resource sharing

✓ Set up CloudWatch alarms for root login, IAM changes, and SG modifications

■ Never create long-lived access keys for applications — use IAM roles instead

■ Never use AdministratorAccess policy on application service roles

■ Never store secrets in ECS environment variables, SSM plain text params, or code

■ Never allow iam:PassRole without constraining to specific role ARNs

■ Avoid wildcard (*) actions and wildcard (*) resources in IAM policies

02-QR Quick Reference & Interview Q&A

Concept	Key Fact
IAM scope	Global service — not region-specific
Default permission	Implicit DENY — everything denied unless explicitly allowed
Explicit DENY	Overrides any number of ALLOWs — always wins
IAM Role temp creds	STS-issued: 15 minutes to 36 hours TTL
Secrets Manager cost	$0.40/secret/month + $0.05/10K API calls
KMS CMK cost	$1/key/month + $0.03/10K API calls
KMS key rotation	Annual automatic rotation when enableKeyRotation = true
GuardDuty cost	~$1-4/month for small account, based on CloudTrail/DNS/VPC logs
Permission boundary	Max permissions ceiling — even if policy allows, boundary caps it
SCP evaluation	Evaluated before identity-based policies — overrides them
Access Analyzer	Free; finds resources accessible outside account/organization

Top Interview Questions

Question	Answer
How does IAM evaluate permissions?	Default deny. Checks: explicit deny (instant block) → SCP deny → resource-based allow (cross
Difference between IAM User, Group, and Role?	User: permanent identity for person/system with credentials. Group: collection of users sharing
Inline policy vs Customer Managed policy?	Inline: directly embedded in one entity, deleted with entity, not reusable. Customer Managed: st
How to give Lambda access to S3?	Create IAM execution role with AmazonS3ReadOnlyAccess (or custom bucket-specific policy).
How do you prevent access key leaks?	Use IAM roles for services, OIDC for CI/CD. Enable GuardDuty credential exposure finding. Us
What is STS and when to use it?	Security Token Service issues temporary credentials. Use for: cross-account access (AssumeR
KMS Envelope Encryption explained?	KMS generates a data key (plaintext + encrypted). App uses plaintext key to encrypt data, then